PROGRAM 3: ERROR HANDLING, STATIC RESOURCES AND RATE-LIMITING

COMMANDS TO RUN

```
npm init -y
npm install express express-rate-limit
node app.js
```

SOURCE CODE

```
// --- public/index.html ---
<!DOCTYPE html>
<html>
<head>
<title>Express Static Resource Demo</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1>Welcome to Express.js</h1>
<p>This page demonstrates static resources.</p>
<img src="image.jpg" width="300">
</body>
</html>

// --- public/style.css ---
body{
font-family: Arial;
text-align: center;
}
h1{
color: blue;
}
p{
color: green;
}

// --- app.js ---
const express = require('express');
const rateLimit = require('express-rate-limit');
const app = express();
app.use(express.static('public'));
const limiter = rateLimit({
windowMs: 1 * 60 * 1000,
max: 5,
message: 'Too many requests, please try again later.'
});
app.use(limiter);
app.get('/hello', (req, res) => {
res.send('Hello Express.js');
});
app.use((req, res) => {
res.status(404).send('Page Not Found');
});
app.listen(3000, () => {
console.log('Server running at http://localhost:3000');
});
```

OUTPUT & TESTING

```
GET http://localhost:3000/
-> Displays index.html with CSS and image (Status: 200 OK)

GET http://localhost:3000/hello
-> Hello Express.js (Status: 200 OK)

GET http://localhost:3000/xyz
-> Page Not Found (Status: 404 Not Found)

After 5 requests in 1 minute:
-> Too many requests, please try again later. (Status: 429)
```

PROGRAM 4: HIERARCHICAL ROUTING IN EXPRESS.JS

COMMANDS TO RUN

```
npm init -y
npm install express
node app.js
```

SOURCE CODE

```
// --- routes/college.js ---
const express = require('express');
const router = express.Router();
const studentsRouter = require('./students');
const marksRouter = require('./marks');
router.use((req, res, next) => {
console.log('College middleware:', req.method, req.originalUrl);
next();
});
router.use('/students', studentsRouter);
router.use('/marks', marksRouter);
module.exports = router;

// --- routes/students.js ---
const express = require('express');
const router = express.Router();
const profileRouter = require('./profile');
router.use('/profile', profileRouter);
module.exports = router;

// --- routes/profile.js ---
const express = require('express');
const router = express.Router();
router.use((req, res, next) => {
console.log('Profile middleware');
next();
});
router.get('/view', (req, res) => {
res.send('Student Profile View');
});
router.get('/:id', (req, res) => {
res.send(`Student Profile ID: ${req.params.id}`);
});
module.exports = router;

// --- routes/marks.js ---
const express = require('express');
const router = express.Router();
router.get('/view', (req, res) => {
res.send('Marks View');
});
router.get('/:id', (req, res) => {
res.send(`Marks for Student ID: ${req.params.id}`);
});
module.exports = router;

// --- app.js ---
const express = require('express');
const app = express();
const collegeRouter = require('./routes/college');
app.use('/college', collegeRouter);
app.listen(3000, () => {
console.log('Server running on port 3000');
});
```

OUTPUT & TESTING

```
GET http://localhost:3000/college/students/profile/view
-> Student Profile View

GET http://localhost:3000/college/students/profile/101
-> Student Profile ID: 101

GET http://localhost:3000/college/marks/view
-> Marks View

GET http://localhost:3000/college/marks/101
-> Marks for Student ID: 101

Console Log Output:
College middleware: GET /college/students/profile/view
Profile middleware
```

PROGRAM 5: REQUEST VALIDATION USING EXPRESS-VALIDATOR

COMMANDS TO RUN

```
npm init -y
npm install express express-validator
node app.js
```

SOURCE CODE

```
// --- middleware/validator.js ---
const { body } = require('express-validator');
exports.userValidation = [
body('name').notEmpty().withMessage('Name is required'),
body('email').isEmail().withMessage('Invalid email format'),
body('age').isInt({ min: 18 }).withMessage('Age must be at least 18')
];

// --- routes/userRoutes.js ---
const express = require('express');
const router = express.Router();
const { validationResult } = require('express-validator');
const { userValidation } = require('../middleware/validator');
router.post('/register', userValidation, (req, res) => {
const errors = validationResult(req);
if (!errors.isEmpty()) {
return res.status(400).json({ errors: errors.array() });
}
res.send('Validation Successful');
});
module.exports = router;

// --- app.js ---
const express = require('express');
const app = express();
const userRoutes = require('./routes/userRoutes');
app.use(express.json());
app.use('/user', userRoutes);
app.listen(3000, () => {
console.log('Server running at http://localhost:3000');
});
```

OUTPUT & TESTING

```
POST http://localhost:3000/user/register
Valid JSON: {"name": "John", "email": "john@gmail.com", "age": 25}
Response: Validation Successful (Status: 200 OK)

Invalid JSON: {"name": "", "email": "abc", "age": 15}
Response: Status 400 Bad Request
{"errors": [{"msg": "Name is required"}, {"msg": "Invalid email format"}, {"msg": "Age must be at least 18"}]}
```

PROGRAM 6 & PROGRAM 7: MONGOOSE REST APIS

PROG 6: MONGOOSE POST & GET (CREATE & READ)

COMMANDS TO RUN

```
npm init -y
npm install express mongoose
node app.js
```

SOURCE CODE

```
// --- app.js ---
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
app.set('port', 3000);
mongoose.connect('mongodb://127.0.0.1:27017/CollegeDB')
.then(() => console.log('MongoDB connected'))
.catch(err => console.log(err));
const studentSchema = new mongoose.Schema({
name: String,
usn: String,
branch: String,
semester: Number
});
const Student = mongoose.model('Student', studentSchema);
app.post('/students', async (req, res) => {
try {
const student = new Student(req.body);
await student.save();
res.status(201).json({
message: 'Student created successfully',
student: student
});
} catch (err) {
res.status(500).json({
message: 'Error creating student',
error: err.message
});
}
});
app.get('/students', async (req, res) => {
try {
const students = await Student.find();
res.status(200).json({
message: 'Students retrieved successfully',
students: students
});
} catch (err) {
res.status(500).json({
message: 'Error retrieving students',
error: err.message
});
}
});
app.listen(3000, () => {
console.log('Server running on port 3000');
});
module.exports = app;
```

OUTPUT & TESTING

```
POST http://localhost:3000/students
Body: {"name": "Rahul", "usn": "4VV23IS001", "branch": "ISE", "semest
er": 5}
Response (201 Created):
{"message": "Student created successfully", "student": {"_id": "...",
"name": "Rahul", "usn": "4VV23IS001", "branch": "ISE", "semester":
5}}

GET http://localhost:3000/students
Response (200 OK):
{"message": "Students retrieved successfully", "students": [{"_id":
"...", "name": "Rahul", "usn": "4VV23IS001", "branch": "ISE", "semest
er": 5}]}
```

PROG 7: MONGOOSE PUT & DELETE (UPDATE & DELETE)

COMMANDS TO RUN

```
npm init -y
npm install express mongoose
node app.js
```

SOURCE CODE

```
// --- app.js ---
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/CollegeDB')
.then(() => console.log('MongoDB connected'))
.catch(err => console.log(err));
const studentSchema = new mongoose.Schema({
name: String,
usn: String,
branch: String,
semester: Number
});
const Student = mongoose.model('Student', studentSchema);
app.put('/students/:usn', async (req, res) => {
try {
const student = await Student.findOneAndUpdate(
{ usn: req.params.usn },
req.body,
{ new: true, runValidators: true }
);
if (!student) {
return res.status(404).json({
message: 'Student not found'
});
}
res.status(200).json({
message: 'Student updated successfully',
student: student
});
} catch (err) {
res.status(500).json({
message: 'Error updating student',
error: err.message
});
}
});
app.delete('/students/:usn', async (req, res) => {
try {
const student = await Student.findOneAndDelete({
usn: req.params.usn
});
if (!student) {
return res.status(404).json({
message: 'Student not found'
});
}
res.status(200).json({
message: 'Student deleted successfully',
student: student
});
} catch (err) {
res.status(500).json({
message: 'Error deleting student',
error: err.message
});
}
});
app.listen(3000, () => {
console.log('Server running on port 3000');
});
module.exports = app;
```

OUTPUT & TESTING

```
PUT http://localhost:3000/students/4VV23IS001
Body: {"name": "Rahul Kumar", "branch": "CSE", "semester": 6}
Response (200 OK):
{"message": "Student updated successfully", "student": {"_id": "...",
"name": "Rahul Kumar", "usn": "4VV23IS001", "branch": "CSE", "semeste
r": 6}}

DELETE http://localhost:3000/students/4VV23IS002
Response (200 OK):
{"message": "Student deleted successfully", "student": {"_id": "...",
"name": "xyz", "usn": "4VV23IS002"}}
```